Lab	Type	Practical
1	A	**Design any listed App/Utility with the help of any AI tool. (No Database)** 1. To-Do List 2. Word Counter & Paragraph Generator 3. Random Password Generator 4. HTML Encoder/Decoder 5. Slug Generator 6. Duplicate Word/Line Remover 7. SQL Formatter 8. Image Watermark Tool
2	A A A A A B B B	**Loops, Strings & Arrays** 1. Write a program to change the case of entered character. 2. Write a program to Replace lower case characters to upper case and Vice-versa. 3. Take 2 strings from the user, and validate 2nd string is contains by 1st or not. 4. Find the second largest element from an array. 5. Write a program to create a Simple Calculator for two numbers (Addition, Multiplication, Subtraction, Division) [using elseif ladder & Switch Case] 6. Find the sum of all elements in an array. 7. Count odd and even numbers in an array. 8. Write a program which find out the first and last occurrence of a character and then replace that character with 'D'.

3		**Class, Objects, Array of Objects & Constructor**
	A	1. Write a program to create a class named Faculty with ID, Name, Age, Weight and Height as data members & also create a member function like GetFacultyDetails() and DisplayFacultyDetails().
	A	2. Write a program to create a class named Employee with Emp_ID, Name, Department, Designation and Salary as data members & also create a member function like GetEmpDetails() and DisplayEmpDetails() for five different employee objects.
	A	3. Write a program to calculate Volume of a Cube using constructor.
	A	4. A car dealership wants to display details of premium sports cars. Create a Car object with Make, Model, Year, FuelType and Horsepower details (use Constructor).
	B	5. Write a program to manage a supermarket inventory. Create an **Item** class with Item_Code, Item_Name, and Stock_Quantity. Use a **constructor** to initialize the objects and an **array of objects** to store and display the details of 10 different items.
	B	6. A clinic management system needs to store patient records. Create a program with a Patient class containing Patient_ID, Name, Age, and Disease. Use an array of objects to accept details for multiple patients and display a formatted summary report of all patients.
4		**Inheritance, Interface & Abstraction**
	A	1. Create a base class Staff and a derived class Doctor to calculate total salary based on a basic pay and specialized doctor allowances.
	A	2. Create an abstract class Billing with an abstract method CalculateBill(). Implement this in InPatientBilling and OutPatientBilling classes.
	A	3. Create an interface INotificationService and implement it across an EmailNotification and SMSNotification module.
	B	4. Create a base class MedicalEquipment and a derived class DiagnosticScanner to calculate maintenance costs based on a baseline service fee and specialized calibration allowances.
	B	5. Create an abstract class PatientRecord with an abstract method CompileReport(). Implement this in InpatientMedicalRecord and OutpatientMedicalRecord classes.
	B	6. Create an interface IInventoryManager and implement it across GroceryStock and ElectronicStock modules. Use exception handling to manage stock shortages and incorrect product details.

5		**<u>Exception Handling</u>**
	A	1. Write a program to perform division of two numbers entered by the user. Use exception handling to catch and display an appropriate message when the denominator is zero.
	A	2. Write a program to accept a numeric value from the user. Use exception handling to detect and handle invalid input when non-numeric data is entered.
	A	3. Write a program to create a BankAccount class with account number and balance. Use exception handling to display an error message when the withdrawal amount exceeds the available balance.
	A	4. Write a program to create a Student class with student details and marks. Use exception handling to generate an error when marks entered are less than 0 or greater than 100.
	B	5. Write a program to create a base class Employee and a derived class Manager. Use a custom exception to handle cases where the salary entered is negative.
	B	6. Write a program to create a base class Vehicle and a derived class Car. Use exception handling to display an error message when the speed entered exceeds the permitted limit.
	B	7. Write a program to create an interface ITransaction with methods for deposit and withdrawal. Implement the interface in a class and use exception handling to manage invalid withdrawal transactions.
	B	8. Write a program to create an interface IPayment with a method to process payments. Implement the interface in different payment classes and use exception handling to handle invalid payment amounts or transaction failures.

6	A A B B C C	**Collection Classes** 1. Develop a menu-driven console application to manage student records using List<Student>. Implement Add, Display, Search, Update, and Delete functionalities. 2. Implement a Shopping Cart system using List<CartItem> that supports adding items, removing items, viewing cart details, and calculating total amount with discount. 3. Create a Student Grade Book using Dictionary<int, Student> where each student record contains subject-wise marks. Implement features to add/update student details, calculate percentage, and display reports. 4. Develop a Product Catalog and Inventory Management System using Dictionary<string, Product> with features for adding products, updating stock, searching, and generating bills. 5. Implement a Role-Based Permission System using HashSet<string> to manage user permissions. Support role assignment, permission checking, and merging of permissions. 6. Develop core features for a Social Networking application using HashSet<T> to manage user interests and friends. Implement mutual friends finding, interest-based suggestions, and duplicate tag removal.
7	A B	**Create Controllers and Action Methods and Implement CRUD APIs** Develop a Web API for Student and Subject management using in-memory storage with List<Student>. Implement GET endpoints to fetch all students, single student, and their subjects. Pre-load 5 sample student records. Extend the Student & Subject Management system by implementing POST, PUT, and DELETE endpoints for managing student and subject records using in-memory List<T> storage.
8	A	**Working with HTTP Status Codes** Enhance the Student and Subject Management Web API by implementing appropriate HTTP status codes for all operations. Use List<Student> for in-memory storage and ensure correct status codes are returned based on operation success, failure, validation errors, and resource availability.
9	A	**Model Designing and Migrations-I** Design normalized entity structures for Users, Roles, Permissions tables with proper relationships, constraints, and navigation flow for the Student Project Management System.

10		**Model Designing and Migrations-II**
	A	Design normalized entity structures for Projects, Tasks, and Project-Allocation tables with proper relationships, constraints, and navigation flow for the Student Project Management System.
11		**CRUD Using EF Core – I**
	A	Implement Create, Read, Update, and Delete operations for Users, Roles and Permission modules using DbContext.
	B	Implement CRUD operations for Projects, Tasks and Project-Allocation modules using DbContext.
12		**CRUD Using EF Core – II**
	A	Integrate soft delete functionality, duplicate prevention, active/inactive filtering, and profile picture path management with optimized query handling for User module.
	B	Integrate score-based task handling, automatic progress computation, and allocation-based access restrictions for faculties.
13		**Dashboard APIs**
	A	Configure dashboard generation APIs for project summaries, task lists, and student progress details using formatted data retrieval.
	B	Integrate downloadable PDF/Excel report generation with filtering, grouped summaries and date-wise reporting mechanisms.
14		**Pagination, Sorting and Filtering**
	A	Implement pagination using page number and page size parameters for Users, Projects, and Tasks modules.
	B	Construct advanced filtering using task status, project status, score ranges, role-wise filtering, and dynamic sorting operations using LINQ expressions.

15		**Common Response with Appropriate Status Codes**
	A	Design common response wrappers containing status, message, success flag, and response payload structure.
	B	Integrate centralized exception handling middleware with proper handling of validation errors, unauthorized requests, missing resources, and internal server failures.
16		**Apply Routing**
	A	Implement conventional routing and attribute routing for Users, Projects, and Tasks controllers.
	B	Configure advanced routing with route constraints, nested routes, custom route naming, and version-compatible route structures.
17		**Model Binding using Body, Query, Route, Header, and Form**
	A	Implement model binding using request body, query parameters, and route parameters for API operations.
	B	Integrate header binding, form-data handling, optional parameters, and custom request validation mechanisms for complex API requests.
18		**Demonstration of File Upload**
	A	Upload profile images to local storage and store file paths in the Users table.
	B	Integrate file size validation, unique file naming strategies, file replacement handling.
19		**API Versioning and Dapper with Stored Procedures**
	A	Configure URL-based API versioning for Users and Projects modules.
	B	Integrate Dapper with stored procedures for dashboard statistics, reporting modules, and optimized read-only operations.
20		**Fluent Validation – Basic Validations**
	A	Configure validations for required fields, email formats, string lengths, and numeric ranges using Fluent Validation.
	B	Integrate validation pipelines with middleware-driven standardized validation error responses.

21		**Fluent Validation – Advanced Validations**
	A	Apply dependent validation rules for dates, scores, and conditional entity properties.
	B	Integrate database-level validation checks including duplicate email validation, unique project validation, and allocation conflict detection.
22		**Authentication using JWT**
	A	Configure login APIs with JWT token generation after successful credential verification.
	B	Integrate claims handling, password hashing, token expiration management, and authentication middleware configuration.
23		**JWT Authorization and Refresh Token Management**
	A	Configure role-based authorization using JWT authorization attributes for Admin, Faculty, and Student modules. Implement refresh token generation, secure token storage, and access token validation mechanisms for authenticated API communication
	B	Integrate allocation-based authorization restricting faculties to assigned projects, students, and tasks only. Implement logout handling and secure session lifecycle management for API security
24		**RBAC and Policy-Based Authorization**
	A	Configure Roles, Permissions, and Role-Permissions mapping for controlled module accessibility and implement database-driven permission management for different user roles.
	B	Integrate dynamic permission verification middleware, project ownership validation, faculty allocation policies, and task-specific access restrictions using custom policy handlers and advanced authorization rules.

25		**Implement Repository Pattern**
	A	Create a separate repository for User, Role and Permission modules. Move all CRUD operations from controllers into repository classes and access user data only through the repository layer.
	B	Extend the repository by implementing reusable methods such as searching users, retrieving active users, handling soft-deleted users, and creating a Generic Repository structure that can be reused by other modules.
26		**Service Layer Implementation**
	A	Create service interfaces and service classes for Users, Role, and Permission. Move business operations from controllers into services while repositories handle only database operations.
27		**Create Unit Tests Using xUnit and Moq for Service Layer Validation**
	A	Create an xUnit Test Project and write unit tests for the User Service Layer of the Student Project Management System using xUnit framework. Perform the following: a. Write unit tests for UserService class to test methods like CreateUserAsync(), GetUserByIdAsync(), GetAllUsersAsync() and UpdateUserAsync(). b. Write unit tests using [Fact] and [Theory] attributes with [InlineData] for testing various scenarios including valid/invalid inputs and validation rules. c. Write unit tests using appropriate xUnit assertions such as Assert.Equal, Assert.NotNull, Assert.True, Assert.ThrowsAsync, etc. to validate service behavior and responses.
28		**Repository Layer Testing using xUnit and EF Core InMemory**
	A	Create integration tests for the Repository Layer of the Student Project Management System using xUnit framework. Perform the following: a. Write tests for UserRepository, ProjectRepository and TaskRepository to validate CRUD operations (Create, Read, Update, Delete). b. Write tests to verify additional repository methods such as GetProjectsByFacultyAsync(), GetTasksByProjectIdAsync(), GetUsersByRoleAsync() etc. c. Use EF Core InMemory database provider to test persistence logic without affecting the actual database. d. Write tests to validate soft delete functionality, filtering, relationships and data integrity constraints.

29		**<u>Project Hosting using Monster ASP.NET</u>**
	A	Deploy the Student Project Management System to a live hosting environment with database connectivity and production configuration handling.
30		**<u>Custom Middleware for Request Logging and Exception Handling and Dependency Injection</u>**
	A	Develop a custom middleware component in the Student & Subject Management Web API that: a. Logs every incoming HTTP request with its URL and timestamp. b. Catches and handles unhandled exceptions gracefully. c. Returns a consistent error response to the client.
	B	Use Dependency Injection for data access. Create a service layer with different lifetimes and inject it into controllers. Demonstrate the behaviour of Transient, Scoped, and Singleton services through logging and ID tracking.